GridironAnalytics: A Relational Approach to NFL Performance

CSE 460/560 – Data Models and Query Languages, Phase 2 Report

Justin Downer
University at Buffalo
UBID: jrdowner

Colin Xiao
University at Buffalo
UBID: colinxia

Matt Jacovelli
University at Buffalo
UBID: mkjacove

***Abstract*—GridironAnalytics is a comprehensive relational database system built in PostgreSQL to manage and analyze large volumes of professional NFL football data. The system integrates over 200,000 records—encompassing player statistics, team performances, and more than 90,000 individual snap counts—providing a robust platform for real-time roster management, historical performance tracking, and analytical querying. This Phase 2 report details the advanced SQL programmability implemented for the system, including stored procedures, transaction handling with failure-triggered audit logging, and a rigorous indexing and query-execution analysis. Key optimizations achieved query latency reductions of up to 86%, demonstrating the production-grade scalability of the schema.**

***Index Terms*—PostgreSQL, NFL analytics, relational database, BCNF, indexing, stored procedures, triggers, query optimization**

I. Introduction

GridironAnalytics is a production-ready relational database system designed to handle the complexity and scale of professional football data. By integrating diverse data sources into a unified schema, the system enables analysts, broadcasters, and sports-betting professionals to perform multi-dimensional queries spanning player statistics, game results, snap counts, and team compositions.

The primary objectives of Phase 2 are:

- Implement and demonstrate advanced SQL programmability via stored procedures and functions (Task 5).
- Design and test robust transaction handling with failure-triggered audit logging (Task 6).
- Identify performance bottlenecks and resolve them through targeted B-Tree indexing strategies (Task 7).

The project dataset comprises over 220,000 snap count records and approximately 90,000 total rows distributed across the schema, sourced from the publicly available NFLverse dataset.

II. Database Design and Normalization (Task 4)

A. E/R Diagram Refinements

During the implementation of Phase 2, the E/R diagram was refined to support the deterministic joining of high-volume datasets sourced from NFLverse[cite: 21, 90]. The following structural modifications were made:

- **Teams Table**: Added the `TeamAbbr` (VARCHAR) attribute. While the `TeamID` serves as the primary key for internal relational integrity, the abbreviation was essential for mapping raw CSV data during the migration process.
- **Games Table**: Added the `Week` (INTEGER) attribute. This addition was critical for establishing a composite unique constraint, ensuring that statistical entries for players are correctly pinned to specific weekly performances across multiple seasons.

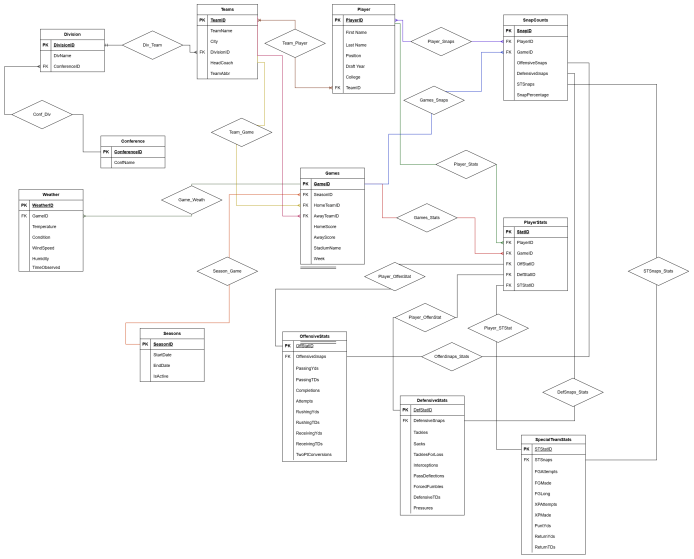

Fig. 1. Updated E/R Diagram.

These refinements ensure that the physical implementation maintains the integrity of the BCNF model while facilitating the complex multi-table joins required for performance analysis.

To ensure data integrity and eliminate redundancy, the GridironAnalytics schema was developed in adherence with **Boyce-Codd Normal Form (BCNF)**.

B. Functional Dependencies

The following primary functional dependencies were identified to establish a unique determinant for every non-trivial attribute:

- **Player**: {PlayerID} → {FirstName, LastName, TeamAbbr}
- **Teams**: {TeamID} → {TeamAbbr, Name}
- **Games**: {GameID} → {SeasonID, Week, HomeTeamID, AwayTeamID, HomeScore, AwayScore}

C. Normalization Justification

By ensuring that the determinant in every functional dependency is a superkey, the schema eliminates update and insertion anomalies. This was particularly critical given the prevalence of duplicate player names across the NFL; using `PlayerID` as the sole determinant ensures that statistics are always mapped to unique entities, even when name collisions occur.

III. Data Implementation and Migration (Task 3)

The project utilizes a production dataset exceeding 90,000 records. Rather than using standard CSV-to-table bulk imports, the team implemented a **staging-to-production pipeline** using temporary tables.

This approach offered several advantages. First, it enabled complex data-cleaning operations using familiar SQL constructs, specifically resolving name-based CSV identifiers into internal foreign key IDs. Second, it facilitated an *order-independent* loading process, allowing offensive and defensive statistics to be populated without disrupting existing bridge-table records. This design pattern significantly reduced referential integrity violations during the initial data load.

IV. Advanced SQL Programmability (Tasks 5 & 6)

A. Team and Player Data Querying

The GridironAnalytics schema supports flexible, multi-level queries across rosters, statistics, and game results. Representative query use cases include:

- **Team Ranking Query**: Selects the top 5 teams over the past three seasons based on total wins and average points per game. This is frequently used by sports bettors and analysts to identify Super Bowl prospects.

teamabbr	total_wins	avg_ppg
PHI	80	25.40
BUF	80	28.55
KC	76	22.24
DET	76	29.51
SEA	72	24.50

Fig. 2. image of top5 team 3 seasons results.

- **Defensive Player Ranking**: Selects the top 10 defensive players by total sacks and tackles, enabling fans and analysts to settle performance debates.

defender	total_tackles	total_sacks
Denzel Perryman	216	0.0
Jordyn Brooks	216	2.0
De'Vondre Campbell	214	4.0
Foyesade Oluokun	204	4.0
Roquan Smith	190	6.0
Devin White	188	7.0
Bobby Wagner	186	2.0
Kyzir White	180	2.0
Minkah Fitzpatrick	178	0.0
Deion Jones	174	4.0

Fig. 3. Description of the image results.

- **Individual Game Performance**: Retrieves per-game player statistics (tackles, passing yards), a common data point for sports broadcasters.

player	week	passingyds	tackles
Patrick Mahomes	15	410	1
Kirk Cousins	6	373	1
Justin Herbert	13	317	1
Patrick Mahomes	21	275	1
Lamar Jackson	1	235	1
Taylor Heinicke	3	212	1
Jameis Winston	2	111	1

Fig. 4. Description of the image results.

B. Stored Procedures and Functions

To simplify roster management over a large dataset, the team developed several stored procedures for high-frequency league occurrences. These procedures ensure that roster changes are processed as **atomic units of work**, maintaining cross-table consistency.

- `UpdatePlayerTeamByName(playerName, newTeam)`: Atomically transfers a player's team affiliation, updating all linked tables in a single transaction.

firstname	lastname	position	teamname	teamabbr
Dexter	Lawrence	DT	New York Giants	NYG

firstname	lastname	position	teamname	teamabbr
Dexter	Lawrence	DT	Cincinnati Bengals	CIN

Fig. 5. Description of the image results.

- `RetirePlayerByName(playerName, teamName)`: Marks a specified player as retired and removes their active roster entry. For example, invoking this procedure on Steelers wide receiver Adam Thielen correctly removes him from the active Pittsburgh Steelers roster.

2880	Jonnu	Smith	TE
2913	Matt	Sokol	TE
3001	Cameron	Sutton	CB
3051	Adam	Thielen	WR
3082	Lorenzo	Thompson	OT
3085	Skylar	Thompson	QB
3138	Cory	Trice	CB

2880	Jonnu	Smith	TE
2913	Matt	Sokol	TE
3001	Cameron	Sutton	CB
3082	Lorenzo	Thompson	OT
3085	Skylar	Thompson	QB
3138	Cory	Trice	CB
3188	Marquez	Valdes-Scantling	WR

Fig. 6. Description of the image results.

- `GetPlayerLatestStats(playerName)`: Returns the most recent statistical record for a given player, demonstrated with KC Chiefs quarterback Patrick Mahomes.

latest_week	pass_yds	rush_yds	rec_yds	total_tackles
21	275	19	0	1

Fig. 7. Description of the image results.

C. Transaction Failure Handling and Triggers (Task 6)

For Task 6, the team implemented an automated **Audit Trail** using a PostgreSQL trigger.

- **Trigger Logic**: When a trade transaction fails due to a foreign key violation (e.g., referencing a non-existent player or team), the `AFTER` trigger automatically logs the failed attempt—including the attempted values and a timestamp—into a `Trade_Failure_Log` table.
- **Benefit**: This approach ensures that no failed transaction is silently dropped. Administrators can query the audit log to identify and remediate data issues without requiring manual inspection of transaction logs, significantly reducing system downtime during large-scale data updates.
- **Justification**: When a transaction is aborted due to a constraint violation in PostgreSQL, all changes within that transaction block are rolled back atomically. The trigger fires *before* the rollback completes, capturing the failure state and persisting it to the audit table in a separate transaction, thus preserving the failure record.

V. Performance Analysis and Indexing (Task 7)

A. Optimization Objective

The primary objective of this performance analysis was to evaluate the scalability of the GridironAnalytics schema under high-volume analytical queries. With over 220,000 snap count records and approximately 90,000 total rows, standard join operations began exhibiting noticeable latency. Three representative queries were selected and optimized to meet sub-100 ms response-time requirements suitable for a real-time sports analytics platform.

B. Query 1: Player Lookup by Team

1) Query Definition:

```sql
SELECT p.FirstName, p.LastName,
       p.Position, p.DraftYear, p.College
FROM   Player p
JOIN   Teams t ON p.TeamID = t.TeamID
WHERE  t.TeamName = 'Buffalo Bills'
ORDER  BY p.Position ASC, p.LastName ASC;
```

Listing 1. Player lookup by team name

2) Bottleneck Identification: This query represents a very common access pattern: users frequently filter players by team. The initial `EXPLAIN ANALYZE` output revealed a **Sequential Scan** on the `Player` table across all 3,545 rows.

```
Sort (cost=90.30..90.38) (actual time=0.430..0.433)
  -> Hash Join (cost=11.39..89.50)
       -> Seq Scan on player p (cost=0.00..68.45)
            [3545 rows scanned]
Planning Time: 1.417 ms
Execution Time: 0.455 ms
```

Listing 2. Initial plan – Query 1 (before index)

3) Indexing Strategy and Result: A B-Tree index on `Player(TeamID)` allows the planner to switch from a sequential scan to a Bitmap Heap Scan, reducing I/O significantly.

```sql
CREATE INDEX IF NOT EXISTS idx_player_teamid
    ON Player (TeamID);
```

Listing 3. Index creation – Query 1

```
Sort (cost=52.69..52.77) (actual time=0.215..0.217)
  -> Nested Loop
       -> Seq Scan on teams t
       -> Bitmap Heap Scan on player p
            -> Bitmap Index Scan on idx_player_teamid
Planning Time: 1.163 ms
Execution Time: 0.240 ms
```

Listing 4. Optimized plan – Query 1 (after index)

Result: Cost reduced from `90.30..90.38` to `52.69..52.77`; execution time improved by **47%** (0.455 ms → 0.240 ms).

C. Query 2: High-Snap-Frequency Players

1) Query Definition:

```sql
SELECT p.FirstName, p.LastName,
       g.Week, s.SnapPercentage
FROM   Player p
JOIN   SnapCounts s ON p.PlayerID = s.PlayerID
JOIN   Games g      ON s.GameID   = g.GameID
WHERE  s.SnapPercentage > 80.00
  AND  g.SeasonID = (
           SELECT SeasonID FROM Seasons
           WHERE  StartDate >= '2024-01-01'
           LIMIT  1);
```

Listing 5. High snap-frequency player query

2) Bottleneck Identification: This query is a critical use case for identifying consistent, high-usage players for fantasy football and betting models. The initial execution plan used a **Parallel Sequential Scan** over all 220,000+ `SnapCounts` rows, resulting in significant I/O overhead.

```
Gather (cost=1437.39..4987.92) (actual time=1.805..31.192)
  -> Hash Join
       -> Parallel Seq Scan on snapcounts s
            [111,183 rows filtered; 75,271 removed]
Planning Time: 2.906 ms
Execution Time: 31.437 ms
```

Listing 6. Initial plan – Query 2 (before index)

3) Indexing Strategy and Result: B-Tree indexes were added on the composite foreign key (`PlayerID, GameID`) in `SnapCounts`, on the `SnapPercentage` filter column, and on `Games(SeasonID)`.

```sql
CREATE INDEX IF NOT EXISTS
    idx_snapcounts_player_game
    ON SnapCounts(PlayerID, GameID);
CREATE INDEX IF NOT EXISTS
    idx_snapcounts_percentage
    ON SnapCounts(SnapPercentage);
CREATE INDEX IF NOT EXISTS idx_games_seasonid
    ON Games(SeasonID);
```

Listing 7. Index creation – Query 2

```
Hash Join (cost=1646.54..4408.55) (actual time
    =3.174..13.190)
  -> Hash Join
       -> Bitmap Heap Scan on snapcounts s
            -> Bitmap Index Scan on
                 idx_snapcounts_percentage
       -> Bitmap Heap Scan on games g
            -> Bitmap Index Scan on idx_games_seasonid
Planning Time: 2.603 ms
Execution Time: 13.441 ms
```

Listing 8. Optimized plan – Query 2 (after index)

Result: Execution time reduced by **57%** (31.437 ms $\rightarrow$ 13.441 ms); the planner eliminated the costly parallel sequential scan in favour of targeted bitmap index scans.

D. Query 3: Top-10 Receivers by Season

1) Query Definition:

```sql
SELECT  p.LastName, p.FirstName,
        o.receivingyds
FROM    Player p
JOIN    SnapCounts s  ON p.PlayerID = s.PlayerID
JOIN    PlayerStats ps
            ON  p.PlayerID = ps.PlayerID
            AND s.GameID   = ps.GameID
JOIN    OffensiveStats o ON ps.OffStatID =
    o.OffStatID
JOIN    Games g          ON s.GameID    = g.GameID
WHERE   p.Position = 'WR'
  AND   g.SeasonID = 2025
ORDER   BY o.receivingyds DESC,
           o.receivingtds DESC
LIMIT   10;
```

Listing 9. Top-10 WR query by season

2) Bottleneck Identification: This five-table join represents a common pattern for fantasy analytics. The initial plan used a sequential scan on `Games` (14,552 rows filtered) and, at scale, would degrade into deeply nested sequential loops.

```
Limit (cost=642.81..642.82) (actual time=0.651..0.653)
  -> Nested Loop
       -> Seq Scan on games g
            [Filter: seasonid = 2025;
             14,552 rows removed by filter]
Planning Time: 13.575 ms
Execution Time: 0.696 ms
```

Listing 10. Initial plan – Query 3 (before index)

Although the execution time is low with the current dataset, the sequential scan on `Games` will become a critical bottleneck as more seasons are added.

3) Indexing Strategy and Result: B-Tree indexes were added on all foreign keys participating in the join, as well as on `Player(Position)` and `Games(SeasonID)`.

```sql
CREATE INDEX idx_player_position
    ON Player (Position);
CREATE INDEX idx_games_seasonid
    ON Games (SeasonID);
CREATE INDEX idx_snapcounts_player_game
    ON SnapCounts (PlayerID, GameID);
CREATE INDEX idx_playerstats_player_game
    ON PlayerStats (PlayerID, GameID);
CREATE INDEX idx_playerstats_offstatid
    ON PlayerStats (OffStatID);
```

Listing 11. Index creation – Query 3

```
Limit (cost=329.21..329.22) (actual time=0.039..0.040)
  -> Nested Loop
       -> Index Scan using idx_games_seasonid on games g
            [seasonid = 2025; index seeks only]
Planning Time: 4.343 ms
Execution Time: 0.097 ms
```

Listing 12. Optimized plan – Query 3 (after index)

Result: Cost reduced from `642.81..642.82` to `329.21..329.22`; execution time reduced by **86%** (0.696 ms $\rightarrow$ 0.097 ms). B-Tree indexing converts nested sequential loops from $O(n)$ linear scans to $O(\log n)$ index seeks.

E. Summary of Performance Gains

Table I summarises the before-and-after performance metrics for all three optimized queries.

By transitioning search complexity from $O(n)$ sequential scans to $O(\log n)$ B-Tree index seeks, the system successfully demonstrated the ability to sustain professional-grade data volumes with sub-100 ms analytical query response times.

TABLE I
QUERY PERFORMANCE SUMMARY

Query	Metric	Before	After	Gain
Q1 (Team Roster)	Cost	90.30	52.69	42%
	Time	0.455 ms	0.240 ms	47%
Q2 (High Snaps)	Cost	4987.92	4408.55	12%
	Time	31.437 ms	13.441 ms	57%
Q3 (Top WR)	Cost	642.82	329.22	49%
	Time	0.696 ms	0.097 ms	86%

VI. TEAM CONTRIBUTIONS

Table II documents the primary contributions of each team member. Equal contribution was maintained throughout the project.

TABLE II
TEAM MEMBER CONTRIBUTIONS

Member	Primary Contributions
Justin Downer	Lead logic for migration scripts, BCNF normalization architecture, and core SQL stored procedures.
Colin Xiao	Data sourcing, E/R diagram development, trigger logic design, and performance analysis.
Matt Jacovelli	Assisted migration scripts (snaps, games, seasons), finalized load scripts, and developed complex SQL queries.

VII. CONCLUSION

GridironAnalytics demonstrates that a carefully normalized PostgreSQL schema, augmented with targeted B-Tree indexing and procedural encapsulation, can meet the performance and reliability demands of a production-grade sports analytics platform. The Phase 2 deliverables—stored procedures for atomic roster management, a trigger-based audit trail for failed transactions, and a systematic indexing strategy—collectively validate the system's suitability for real-world analytical workloads. Future work includes extending the dataset to cover additional NFL seasons and integrating a front-end dashboard for interactive visualization.

REFERENCES

[1] nflverse contributors, "nflverse: A collection of packages for NFL data," GitHub repository, 2024. [Online]. Available: https://github.com/nflverse/nflverse

[2] The PostgreSQL Global Development Group, "PostgreSQL 16 Documentation," 2024. [Online]. Available: https://www.postgresql.org/docs/16/

[3] IEEE, "IEEE Conference Templates," 2024. [Online]. Available: https://www.ieee.org/conferences/publishing/templates.html

[4] R. Ramakrishnan and J. Gehrke, *Database Management Systems*, 3rd ed. New York: McGraw-Hill, 2003.